Chapter 16

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) [208] has emerged as one of the most practically impactful techniques for deploying large language models in production. Rather than relying solely on knowledge encoded in model weights during training, RAG equips LLMs with a dynamic, updatable external memory—enabling accurate, grounded, and verifiable responses across a wide range of knowledge-intensive tasks.

16.1 Motivation and Problem Statement

Why LLMs Need External Knowledge

Large language models store knowledge *parametrically*—compressed into billions of weights during training. This creates three fundamental limitations:

1. **Hallucination**: Models confidently generate plausible-sounding but factually incorrect statements when queried beyond their reliable knowledge boundary.
2. **Knowledge Staleness**: Training data has a cutoff date; models cannot know about events, papers, or product updates that occurred after training.
3. **Domain Specificity**: General-purpose models lack deep knowledge of proprietary codebases, internal documents, specialized regulations, or enterprise data.

16.1.1 Parametric vs. Non-Parametric Knowledge

We can formalize the distinction between the two knowledge sources. Let $\mathcal{M}_\theta$ denote a language model with parameters θ, and let $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ be an external document corpus. The probability of generating answer a given query q under each paradigm is:

$$P_{\text{parametric}}(a \mid q) = P_{\mathcal{M}_\theta}(a \mid q) \tag{16.1}$$

$$P_{\text{RAG}}(a \mid q, \mathcal{D}) = \sum_{d \in \mathcal{D}} P_{\mathcal{M}_\theta}(a \mid q, d)\, P_{\text{ret}}(d \mid q, \mathcal{D}) \tag{16.2}$$

where $P_{\text{ret}}(d \mid q, \mathcal{D})$ is the retrieval distribution over documents. RAG marginalizes over retrieved evidence, grounding generation in non-parametric knowledge.

The Library Analogy

Think of a parametric LLM as a scholar who has memorized an enormous library but graduated years ago. RAG gives that scholar a library card—they can look things up in real time, cite sources, and acknowledge when they need to check a reference rather than guessing from memory.

16.1.2 When to Use RAG vs. Fine-Tuning vs. Long Context

Table 16.1: Decision guide: RAG vs. Fine-Tuning vs. Long Context

Criterion	RAG	Fine-Tuning	Long Context	RAG + FT
Knowledge updates frequently	✓	×	×	✓
Need citations / grounding	✓	×	✓	✓
Proprietary large corpus	✓	×	×	✓
Adapt style / format	×	✓	×	✓
Teach new reasoning skills	×	✓	×	✓
Corpus fits in context window	×	×	✓	×
Low latency required	×	✓	×	×

Common Misconception

RAG is *not* a replacement for fine-tuning. Fine-tuning teaches the model *how* to reason and respond; RAG provides *what* to reason about. They are complementary. A model fine-tuned to follow instructions well will use retrieved context more effectively than a base model.

16.2 Core RAG Architecture

A standard RAG system consists of two phases: an **offline indexing pipeline** that processes and stores documents, and an **online retrieval-generation pipeline** that serves queries.

16.2.1 Full Pipeline Diagram

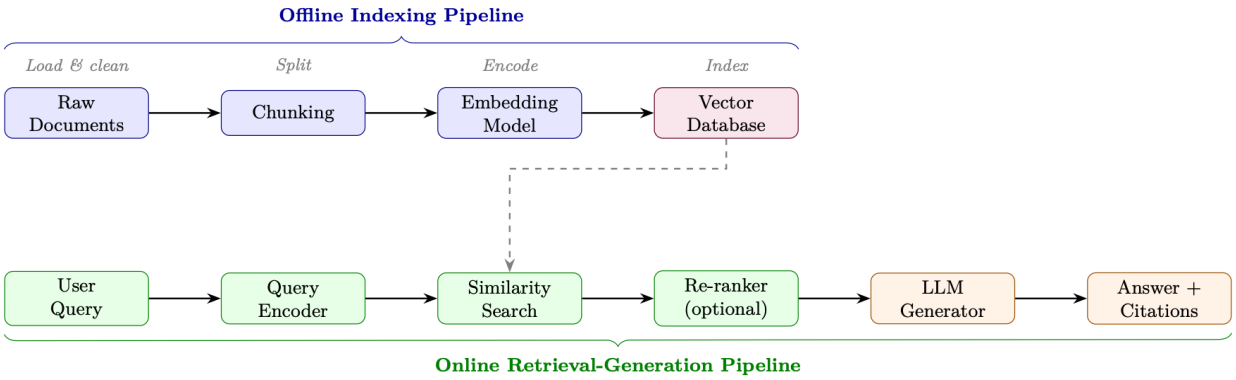

Figure 16.1: End-to-end RAG architecture. The offline pipeline (blue) indexes documents once; the online pipeline (green/orange) serves each query at inference time.

16.2.2 Indexing Pipeline

Document Loading. Documents arrive in heterogeneous formats (PDF, HTML, Markdown, DOCX, code). Loaders extract clean text and preserve metadata (source URL, page number, section title, timestamp) that will be stored alongside embeddings for filtering and citation.

Chunking. Long documents must be split into chunks that fit within the embedding model's context window (typically 512 tokens) and are semantically coherent. Chunking strategy is one of the highest-impact decisions in RAG system design (see Section 16.4).

Embedding. Each chunk c_i is encoded into a dense vector $\mathbf{e}_i = f_\phi(c_i) \in \mathbb{R}^d$ using an embedding model f_ϕ. These vectors are stored in a vector database alongside the original text and metadata.

16.2.3 Retrieval

Given a query q, the retrieval step encodes it as $\mathbf{q} = f_\phi(q)$ and finds the k most similar chunks by cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{e}_i) = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \, \|\mathbf{e}_i\|} \tag{16.3}$$

The top-k chunks $\mathcal{C}_k = \{c_{(1)}, \ldots, c_{(k)}\}$ are returned as context.

16.2.4 Generation

Retrieved chunks are injected into a prompt template:

```
SYSTEM_PROMPT = """You are a helpful assistant. Answer the question using ONLY
the provided context. If the context does not contain enough information,
say so explicitly. Cite your sources using [Doc N] notation."""

def build_rag_prompt(query: str, chunks: list[dict]) -> str:
    context_str = "\n\n".join(
        f"[Doc {i+1}] (Source: {c['source']}, Page: {c.get('page','N/A')})\n{c['
    text']}"
        for i, c in enumerate(chunks)
    )
    return f"""{SYSTEM_PROMPT}

Context:
{context_str}

Question: {query}

Answer:"""
```

Listing 16.1: Standard RAG prompt template

16.3 Retrieval Methods

16.3.1 Sparse Retrieval: BM25 and TF-IDF

Sparse retrieval methods represent documents and queries as high-dimensional sparse vectors over the vocabulary. The classic BM25 scoring function [309] for document d given query q with terms $t_1, \ldots, t_n$ is:

$$\text{BM25}(d, q) = \sum_{i=1}^{n} \text{IDF}(t_i) \cdot \frac{f(t_i, d) \cdot (k_1 + 1)}{f(t_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \tag{16.4}$$

where $f(t_i, d)$ is term frequency, $|d|$ is document length, avgdl is average document length, and $k_1 \in [1.2, 2.0]$, $b = 0.75$ are tuning parameters.

When Sparse Retrieval Still Wins

- **Exact keyword matching**: product codes, error codes, proper nouns, rare terms
- **Low-resource domains**: insufficient training data for dense models
- **Interpretability**: easy to debug why a document was retrieved
- **Speed**: no GPU required; scales to billions of documents with inverted indices
- **Out-of-vocabulary terms**: new terminology not seen during embedding training

16.3.2 Dense Retrieval: DPR

Dense Passage Retrieval (DPR) [180] uses two separate BERT-based encoders—a *query encoder* E_Q and a *passage encoder* E_P—trained with contrastive loss to place relevant query-passage pairs close together in embedding space.

Bi-Encoder Architecture.

$$\text{sim}(q, p) = E_Q(q)^\top E_P(p) \tag{16.5}$$

Training with In-Batch Negatives. Given a batch of B query-passage pairs $\{(q_i, p_i^+)\}_{i=1}^B$, the contrastive loss treats all other passages in the batch as negatives:

$$\mathcal{L}_{\text{DPR}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp\Big(E_Q(q_i)^\top E_P(p_i^+)/\tau\Big)}{\sum_{j=1}^{B} \exp(E_Q(q_i)^\top E_P(p_j)/\tau)} \tag{16.6}$$

where τ is a temperature hyperparameter. Hard negatives (passages that are lexically similar but semantically irrelevant) are crucial for training strong retrievers.

Approximate Nearest Neighbor Search. At scale, exhaustive search over millions of embeddings is infeasible. FAISS [171] (Facebook AI Similarity Search) provides efficient approximate nearest neighbor (ANN) search using:

- **IVF (Inverted File Index)**: cluster vectors into Voronoi cells; search only nearby cells
- **HNSW (Hierarchical Navigable Small World)** [248]: graph-based index with $O(\log N)$ search
- **PQ (Product Quantization)**: compress vectors to reduce memory footprint

16.3.3 Hybrid Retrieval with Reciprocal Rank Fusion

Hybrid retrieval combines sparse and dense scores. A simple linear combination is:

$$s_{\text{hybrid}}(d, q) = \alpha \cdot s_{\text{dense}}(d, q) + (1 - \alpha) \cdot s_{\text{sparse}}(d, q) \tag{16.7}$$

However, scores from different systems are not directly comparable. **Reciprocal Rank Fusion (RRF)** [62] avoids this by operating on ranks rather than scores:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \tag{16.8}$$

where $\mathcal{R}$ is the set of ranked lists (e.g., BM25 ranking and dense ranking), $\text{rank}_r(d)$ is the rank of document d in list r, and $k = 60$ is a smoothing constant that reduces the impact of very high-ranked documents.

RRF Calculation

Suppose BM25 ranks document d at position 3, and dense retrieval ranks it at position 7. With $k = 60$:

$$\text{RRF}(d) = \frac{1}{60+3} + \frac{1}{60+7} = \frac{1}{63} + \frac{1}{67} \approx 0.0159 + 0.0149 = 0.0308$$

A document ranked 1st in both lists would score $\frac{1}{61} + \frac{1}{61} \approx 0.0328$.

16.3.4 Learned Sparse Retrieval: SPLADE and SPLADEv2

Why SPLADE?

Traditional sparse retrieval (BM25) relies on exact lexical matching — it fails when the query says "car" but the document says "automobile." Dense retrieval (DPR) captures semantics but loses interpretability, requires GPU at query time, and produces large indexes. **SPLADE** gets the best of both worlds: sparse vectors (fast inverted-index lookup like BM25) with learned semantic expansion (handles synonyms and related concepts like dense models).

SPLADE (v1) — Core Idea. SPLADE (Sparse Lexical and Expansion Model) [95] uses a pre-trained masked language model (e.g., BERT/DistilBERT) to produce a sparse vector over the *entire vocabulary* for each document or query. The key insight: the MLM head already knows which words are semantically related to each position in a text — SPLADE repurposes this knowledge as term importance weights.

Architecture. Given input text $x = [x_1, \ldots, x_n]$:

1. Pass through a transformer encoder to get contextual representations $\mathbf{H} \in \mathbb{R}^{n \times |\mathcal{V}|}$ via the MLM head
2. Aggregate across positions and apply a saturating activation:

$$w_t(x) = \log\left(1 + \text{ReLU}\left(\max_{i \in [1,n]} \mathbf{H}_i[t]\right)\right) \tag{16.9}$$

where $\mathbf{H}_i[t]$ is the MLM logit for vocabulary token t at input position i.

- The $\log(1 + \cdot)$ saturation prevents any single term from dominating (similar to TF saturation in BM25)
- The ReLU ensures sparsity — most vocabulary terms get weight zero
- The max pooling across positions captures the strongest signal for each term from any position in the text
- **Expansion**: Even tokens *not present* in the original text can get non-zero weight (e.g., a document about "neural networks" may get weight for "deep learning," "AI," "backpropagation")

Scoring. Query and document are each mapped to sparse vectors $\mathbf{w}^q, \mathbf{w}^d \in \mathbb{R}^{|\mathcal{V}|}$. The relevance score is a simple dot product:

$$s(q, d) = \sum_{t \in \mathcal{V}} w_t^q \cdot w_t^d \tag{16.10}$$

Because both vectors are sparse (typically 20–200 non-zero entries out of 30K vocabulary), this can be computed efficiently using standard inverted indexes (Lucene, Anserini) — no GPU needed at query time.

Training. SPLADE is trained with contrastive learning (in-batch negatives + hard negatives) plus two regularization terms:

$$\mathcal{L} = \mathcal{L}_{\text{contrastive}} + \lambda_q \|\mathbf{w}^q\|_1 + \lambda_d \|\mathbf{w}^d\|_1 \tag{16.11}$$

The L_1 penalties on query and document representations encourage sparsity — without them, the model would learn dense representations that defeat the purpose.

SPLADEv2 — Key Improvements. SPLADEv2 [94] introduces several refinements that significantly improve efficiency and effectiveness:

1. **Distillation from cross-encoder**: Instead of training only on binary relevance labels, SPLADEv2 uses a cross-encoder teacher (e.g., MonoT5 [269]) to provide soft relevance scores. This gives richer training signal:

$$\mathcal{L}_{\text{distill}} = \text{KL}(\sigma(s_{\text{student}}) \,\|\, \sigma(s_{\text{teacher}})) \tag{16.12}$$

2. **Separate query/document encoders**: SPLADEv2 uses different sparsity targets for queries vs. documents. Queries are encouraged to be *more sparse* (faster lookup) while documents can be slightly denser (pre-computed offline):

$$\lambda_q > \lambda_d \quad (\text{e.g., } \lambda_q = 3 \times 10^{-4},\ \lambda_d = 1 \times 10^{-4}) \tag{16.13}$$

3. **FLOPS regularization**: Instead of simple L_1, SPLADEv2 introduces a FLOPS-aware regularizer that directly penalizes the expected retrieval cost:

$$\mathcal{L}_{\text{FLOPS}} = \sum_{t \in \mathcal{V}} (\overline{a}_t^q)^2 + \sum_{t \in \mathcal{V}} \left(\overline{a}_t^d\right)^2 \tag{16.14}$$

   where $\overline{a}_t$ is the mean activation for term t across the batch. This penalizes terms that are non-zero for many documents (high posting list length = slow retrieval).

4. **Efficient backbone**: Uses DistilBERT (66M params) instead of BERT-base (110M), halving encoding time with minimal quality loss.

SPLADE vs. SPLADEv2 Comparison

Aspect	SPLADE (v1)	SPLADEv2
Training signal	Binary relevance + hard negatives	Cross-encoder distillation
Sparsity control	L_1 regularization	FLOPS-aware regularization
Query/doc symmetry	Same encoder, same λ	Asymmetric (sparser queries)
Backbone	BERT-base (110M)	DistilBERT (66M)
MRR@10 (MS MARCO [18])	34.0	36.8
Avg non-zero terms/doc	∼200	∼120 (40% sparser)

When to Use SPLADE

- **Use SPLADE/v2 when**: You need semantic retrieval without GPU at query time, your infrastructure already has inverted indexes (Elasticsearch, Lucene), or you need interpretable relevance scores (you can inspect which expanded terms matched).
- **Prefer dense retrieval when**: You have GPU budget for query encoding, need multilingual support (dense models transfer better), or your queries are very short (1–2 words where expansion helps less).
- **Best practice**: Use SPLADEv2 as the first-stage retriever + cross-encoder reranker for top-k. This matches or beats dense retrieval pipelines at lower latency.

16.3.5 ColBERT: Late Interaction

ColBERT [182] encodes queries and documents into *sets* of token-level embeddings and uses a *MaxSim* operator for scoring:

$$s(q, d) = \sum_{i \in |\mathbf{q}|} \max_{j \in |\mathbf{d}|} \mathbf{q}_i^\top \mathbf{d}_j \tag{16.15}$$

This late interaction mechanism is more expressive than single-vector bi-encoders while being far faster than cross-encoders, since document embeddings are pre-computed offline.

Architecture. Both the query encoder E_Q and document encoder E_D are BERT-based models that produce *per-token* embeddings (not a single [CLS] vector). Each token embedding is projected to a lower dimension (typically 128) via a linear layer:

$$\mathbf{q}_i = \text{Linear}(E_Q(q)_i) \in \mathbb{R}^{128}, \quad i = 1, \ldots, |q| \tag{16.16}$$

$$\mathbf{d}_j = \text{Linear}(E_D(d)_j) \in \mathbb{R}^{128}, \quad j = 1, \ldots, |d| \tag{16.17}$$

Training. ColBERT is trained with a pairwise softmax cross-entropy loss over positive and negative passages. Given a query q, a positive passage d^+, and a set of negative passages $\{d_1^-, \ldots, d_N^-\}$:

$$\mathcal{L}_{\text{ColBERT}} = -\log \frac{\exp(s(q, d^+))}{\exp(s(q, d^+)) + \sum_{k=1}^{N} \exp(s(q, d_k^-))} \tag{16.18}$$

where $s(q, d)$ is the MaxSim score from Equation 16.15. Negatives are sourced from:

- **In-batch negatives**: Other passages in the same training batch (free, abundant)
- **Hard negatives**: Passages retrieved by BM25 that are lexically similar but semantically irrelevant (most impactful for quality)
- **Distillation negatives** (ColBERTv2 [313]): Use a cross-encoder teacher to mine the hardest negatives and distill its scores into ColBERT

Indexing and Serving. At index time, all document token embeddings are pre-computed and stored (with optional compression via residual quantization in ColBERTv2). At query time, only the query tokens are encoded live, and MaxSim is computed against the stored document embeddings. This separation enables:

- **Offline document encoding**: Encode once, serve many queries
- **PLAID indexing** [313]: Cluster document embeddings, use centroids for initial candidate retrieval, then compute exact MaxSim only on candidates—reducing latency by 5–10×
- **Index size**: $|d| \times 128$ floats per document (larger than single-vector methods but compressible to ∼2 bytes/dimension with quantization)

16.3.6 Retrieval Method Comparison

Table 16.2: Comparison of retrieval methods across key dimensions

Method	Latency	Accuracy	Index Size	GPU	Best For
TF-IDF [336]	Very Low	Low	Small	No	Baseline, exact match
BM25 [309]	Very Low	Medium	Small	No	Keyword search, rare terms
DPR / bi-encoder [180]	Low	High	Large	Yes	Semantic similarity
SPLADE [95]	Low	High	Medium	Yes	Hybrid accuracy + speed
ColBERT [182]	Medium	Very High	Very Large	Yes	High-accuracy retrieval
Cross-encoder [268]	High	Highest	N/A	Yes	Re-ranking top-k
Hybrid (RRF) [62]	Low	Very High	Large	Yes	Production systems

16.4 Chunking Strategies

Chunking is the process of splitting documents into segments that are (1) small enough to fit in an embedding model's context window, (2) semantically coherent, and (3) contain enough context to be useful when retrieved in isolation.

16.4.1 Fixed-Size Chunking with Overlap

The simplest strategy: split every W tokens with an overlap of O tokens between consecutive chunks.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,       # tokens per chunk
    chunk_overlap=64,     # overlap to preserve context across boundaries
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""]
)
chunks = splitter.split_documents(documents)
```

Listing 16.2: Fixed-size chunking with overlap

Overlap formula: For a document of length L tokens, the number of chunks is:

$$N_{\text{chunks}} = \left\lceil \frac{L - O}{W - O} \right\rceil \tag{16.19}$$

16.4.2 Semantic Chunking

Rather than splitting at fixed intervals, semantic chunking splits at *topic boundaries* detected by measuring embedding similarity between consecutive sentences:

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

chunker = SemanticChunker(
    embeddings=OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile",  # or "standard_deviation"
    breakpoint_threshold_amount=95,          # split at top 5% dissimilarity
)
chunks = chunker.split_documents(documents)
```

Listing 16.3: Semantic chunking via embedding similarity

16.4.3 Document-Structure-Aware Chunking

For structured documents (Markdown, HTML, code), split at natural boundaries:

- **Markdown**: split at `##` headers, preserving section context
- **HTML**: split at `<section>`, `<article>`, `<p>` tags
- **Code**: split at function/class definitions, preserving imports in each chunk
- **Tables**: keep entire tables as single chunks; never split mid-row

16.4.4 Parent-Child Chunking

A powerful pattern that decouples retrieval granularity from generation context:

1. **Index small child chunks** (e.g., 128 tokens) for precise retrieval
2. **Return large parent chunks** (e.g., 512 tokens) to the LLM for richer context

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000)
child_splitter  = RecursiveCharacterTextSplitter(chunk_size=400)

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=InMemoryStore(),
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever.add_documents(documents)
```

Listing 16.4: Parent-child chunking with LangChain

16.4.5 Empirical Guidelines for Chunk Size

Table 16.3: Chunk size recommendations by use case

Use Case	Recommended Chunk Size	Overlap
Factoid QA (precise facts)	128–256 tokens	20–32 tokens
Summarization / synthesis	512–1024 tokens	64–128 tokens
Code retrieval	Full function	None
Legal / regulatory documents	Paragraph-level	1 sentence
Conversational / chat	256–512 tokens	32–64 tokens

16.5 Advanced RAG Patterns

16.5.1 Query Transformation

Raw user queries are often ambiguous, too short, or poorly matched to document language. Query transformation techniques improve retrieval before the search step.

HyDE (Hypothetical Document Embeddings) [106]. Instead of embedding the query directly, generate a *hypothetical answer* and embed that:

$$\hat{d} = \text{LLM}(q), \quad \mathbf{e}_{\text{query}} = f_\phi(\hat{d}) \tag{16.20}$$

The intuition: a hypothetical answer is in the same linguistic register as real documents, reducing the query-document distribution gap.

Step-Back Prompting. For specific questions, first generate a more general "step-back" question, retrieve for both, and combine the contexts. Example: "What is the boiling point of ethanol at 2 atm?" → step-back: "What factors affect the boiling point of liquids?"

Multi-Query Generation. Generate M diverse reformulations of the query, retrieve for each, and union the results:

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    llm=ChatOpenAI(temperature=0.7),
    include_original=True,   # also retrieve for original query
)
# Internally generates 3 query variants, retrieves for each, deduplicates
docs = retriever.get_relevant_documents(query)
```

Listing 16.5: Multi-query retrieval

16.5.2 Re-Ranking

After initial retrieval of top-k candidates, a *cross-encoder* re-ranker scores each query-document pair jointly (attending to both simultaneously), producing much more accurate relevance scores at the cost of higher latency:

$$s_{\text{cross}}(q, d) = \text{CrossEncoder}([q; d]) \tag{16.21}$$

Cross-encoders cannot be used for first-stage retrieval (no pre-computed document embeddings), but are ideal for re-ranking a small candidate set (typically $k = 20$–100).

```python
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("BAAI/bge-reranker-large")

def rerank(query: str, docs: list[str], top_n: int = 5) -> list[str]:
    pairs = [(query, doc) for doc in docs]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(scores, docs), reverse=True)
    return [doc for _, doc in ranked[:top_n]]
```

Listing 16.6: Cross-encoder re-ranking with BGE

16.5.3 Contextual Compression

Retrieved chunks often contain irrelevant sentences surrounding the relevant passage. Contextual compression uses an LLM to extract only the relevant portions:

```python
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vectorstore.as_retriever()
)
compressed_docs = compression_retriever.get_relevant_documents(query)
```

Listing 16.7: LLM-based contextual compression

16.5.4 Self-RAG

Self-RAG [13] trains a single model to (1) decide *whether* to retrieve, (2) generate with or without retrieval, and (3) *critique* its own output using special reflection tokens:

- `[Retrieve]`: should the model retrieve additional passages?
- `[IsRel]`: is the retrieved passage relevant to the query?
- `[IsSup]`: does the generated statement follow from the retrieved passage?
- `[IsUse]`: is the overall response useful?

The model is trained end-to-end to predict these tokens alongside the response, enabling fine-grained control over retrieval and self-grading.

16.5.5 CRAG: Corrective RAG

CRAG [401] adds a *retrieval evaluator* that grades retrieved documents and triggers corrective actions:

1. Retrieve top-k documents
2. Grade each document: **Correct** / **Ambiguous** / **Incorrect**
3. If all documents are incorrect or ambiguous → fall back to web search
4. If some documents are correct → use knowledge refinement (strip irrelevant sentences)
5. Generate answer from refined context

16.5.6 Adaptive RAG

Adaptive RAG [162] routes queries to different retrieval strategies based on predicted complexity:

- **No retrieval**: simple factual queries the model can answer from parameters
- **Single-step RAG**: standard retrieve-then-generate for moderate queries
- **Multi-step RAG**: iterative retrieval for complex multi-hop questions

A lightweight classifier trained on query complexity labels routes each incoming query.

16.5.7 Graph RAG

Microsoft's Graph RAG [82] constructs a *knowledge graph* from the document corpus and uses community detection to generate hierarchical summaries:

1. **Entity extraction**: LLM extracts entities and relationships from each chunk
2. **Graph construction**: build a graph $G = (V, E)$ where nodes are entities and edges are relationships
3. **Community detection**: apply Leiden algorithm to find communities at multiple resolutions
4. **Community summaries**: LLM generates a summary for each community
5. **Query**: for global queries, map-reduce over community summaries; for local queries, use standard vector search

When to Use Graph RAG

Graph RAG excels at *global* queries that require synthesizing information across many documents ("What are the main themes in this corpus?") but is expensive to build and maintain. Standard RAG is better for *local* queries ("What did document X say about topic Y?").

16.5.8 RAG-Fusion

RAG-Fusion [300] generates multiple search queries from the original, retrieves for each, and fuses the ranked lists using RRF (Equation 16.8):

```
def reciprocal_rank_fusion(ranked_lists: list[list[str]], k: int = 60) -> list[str
    ]:
    """Fuse multiple ranked document lists using RRF."""
    scores: dict[str, float] = {}
    for ranked in ranked_lists:
        for rank, doc_id in enumerate(ranked, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

def rag_fusion(query: str, retriever, llm, n_queries: int = 4) -> str:
    # Step 1: Generate query variants
    variants = generate_query_variants(query, llm, n=n_queries)
```

```
    # Step 2: Retrieve for each variant
    all_ranked = [retriever.retrieve(q) for q in [query] + variants]
    # Step 3: Fuse with RRF
    fused_docs = reciprocal_rank_fusion(all_ranked)
    # Step 4: Generate answer
    return generate_answer(query, fused_docs[:5], llm)
```

Listing 16.8: RAG-Fusion with RRF

16.6 Efficient RAG Decoding: REFRAG

A practical bottleneck of RAG is *decoding latency*: the retrieved passages concatenated into the LLM context are often long yet sparsely relevant, inflating time-to-first-token (TTFT) and KV-cache memory. REFRAG [220] observes that because retrieved passages are independently sourced (via diversity or deduplication during re-ranking), their attention patterns are *block-diagonal*—most cross-passage attention is near zero. This sparsity means that the majority of computations over the RAG context during decoding are unnecessary.

Compress–Sense–Expand Framework. REFRAG exploits this structure via a three-phase decoding strategy:

1. **Compress**: Replace full KV representations of retrieved passages with compact summaries (e.g., mean-pooled keys/values per passage block), drastically reducing memory.
2. **Sense**: At each decoding step, use lightweight attention over the compressed representations to identify which passage blocks are relevant to the current token.
3. **Expand**: Reconstruct full KV entries only for the selected blocks, performing exact attention over the sparse active set.

Results. On LLaMA-based models, REFRAG achieves up to 30.85× TTFT speedup (a 3.75× improvement over prior sparse-attention baselines) with no loss in perplexity. It also extends effective context length by 16× under fixed memory budgets. These gains hold across RAG, multi-turn conversation, and long-document summarization tasks.

Why REFRAG Matters for Agentic RAG

Agentic RAG (Section 16.7) requires *multiple* retrieval rounds per query, compounding latency. Efficient decoding methods like REFRAG are essential infrastructure: they make iterative retrieve-reason-generate loops practical at scale by ensuring each round's decoding cost is sublinear in context length.

16.7 Agentic RAG

16.7.1 Motivation: Limits of Static RAG

Standard RAG follows a fixed retrieve-then-generate pattern. This fails on:

- **Multi-hop questions**: "Who founded the company that acquired OpenAI's main competitor in 2023?" requires chaining multiple retrievals
- **Ambiguous queries**: the right retrieval strategy depends on what is found
- **Heterogeneous sources**: different sub-questions require different knowledge bases
- **Iterative refinement**: initial retrieval may reveal that a different query is needed

RAG as a Markov Decision Process

Agentic RAG frames retrieval as a sequential decision problem. The *state* is the current context (query + retrieved documents so far); the *actions* include retrieve, reason, generate, and stop; the *reward* is answer correctness. The agent learns a policy for when and what to retrieve.

16.7.2 Agentic RAG Architecture

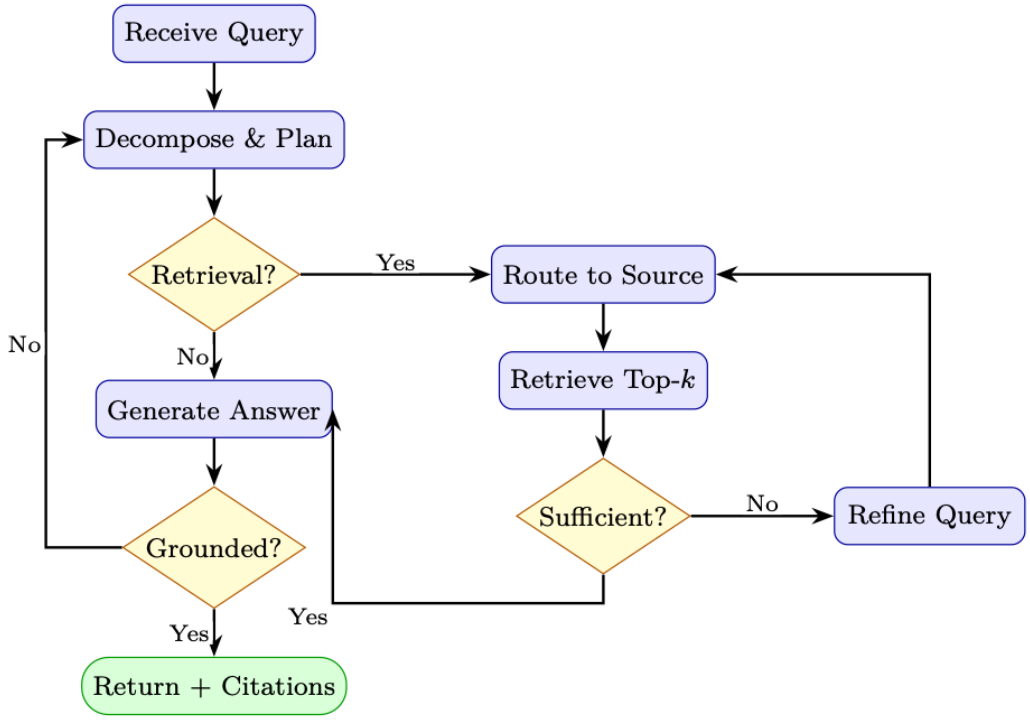

Figure 16.2: Agentic RAG control flow. The agent iteratively plans, retrieves, evaluates sufficiency, and self-checks grounding before returning an answer.

16.7.3 Multi-Source Routing

An agentic RAG system can route sub-queries to specialized knowledge sources. The core insight is that different question types demand different retrieval backends—no single index excels at everything.

Why Route? Consider a financial analyst's assistant handling four queries:

- "What is our company's PTO policy?" → **Vector DB** (internal documents)
- "What did the Fed announce yesterday?" → **Web search** (real-time)
- "Show Q3 revenue by region" → **SQL database** (structured data)
- "How does our auth middleware validate tokens?" → **Code index** (codebase)

A flat retrieve-from-one-index approach either misses the answer or returns irrelevant passages. Routing selects the *right tool for the right sub-question* before retrieval begins.

Routing Strategies. Three main approaches, in increasing sophistication:

1. **Rule-based routing.** Keyword triggers (e.g., SQL keywords → database, URL patterns → web). Fast and interpretable but brittle for ambiguous queries.

2. **Classifier-based routing.** A lightweight model (e.g., a fine-tuned BERT classifier or logistic regression over query embeddings) predicts the best source. Low latency (<10 ms) and trainable on routing logs, but requires labeled data.

3. **LLM-based routing.** The LLM itself decides the source in a structured-output call (see Listing below). Most flexible—handles novel query types and can explain its reasoning—but adds one LLM call of latency.

Router as a Learned Policy

Multi-source routing is a *classification* problem at its simplest and a *planning* problem at its richest. When treated as an RL policy—where the state is the query plus conversation history, the action is the choice of source (and optional query rewrite), and the reward is downstream answer quality—the router can be optimized end-to-end via policy gradient techniques (Chapter 8).

Practical Considerations.

- **Fallback chains**: If the primary source returns low-confidence results, try the next-best source.
- **Parallel fan-out**: For ambiguous queries, retrieve from multiple sources simultaneously and fuse results via Reciprocal Rank Fusion (Table 16.2).
- **Cost awareness**: Web search and API calls may have monetary cost or rate limits; the router should factor these in.
- **Observability**: Log every routing decision with its reasoning—essential for debugging and retraining.

```
from enum import Enum
from pydantic import BaseModel

class KnowledgeSource(str, Enum):
    VECTOR_DB   = "vector_db"    # internal documents
    WEB_SEARCH  = "web_search"   # real-time web
    SQL_DB      = "sql_db"       # structured data
    CODE_INDEX  = "code_index"   # codebase
    API         = "api"          # external APIs

class RouteDecision(BaseModel):
    source: KnowledgeSource
    refined_query: str
    reasoning: str

def route_query(query: str, llm) -> RouteDecision:
    """Use LLM to decide which knowledge source to query."""
    prompt = f"""Given the query: "{query}"

Decide which knowledge source to use:
- vector_db: for internal documents, policies, past reports
- web_search: for current events, recent information
- sql_db: for numerical data, statistics, structured records
- code_index: for code examples, API documentation
- api: for real-time data (weather, stock prices, etc.)

Return a JSON with: source, refined_query, reasoning."""

    return llm.with_structured_output(RouteDecision).invoke(prompt)
```

Listing 16.9: Multi-source agentic RAG router

16.7.4 Full Agentic RAG Implementation

The previous sections introduced individual components—routing, retrieval, evaluation. A full agentic RAG system orchestrates these as a *graph of stateful nodes*, where control flow depends on intermediate results. The implementation below uses LangGraph to wire four nodes into a loop:

1. **Plan**: Decompose the user query into sub-queries (one per information need).
2. **Retrieve**: Route each sub-query to the appropriate source and fetch documents.
3. **Evaluate**: Judge whether the accumulated context is sufficient to answer the original query.
4. **Generate**: Synthesize a final answer with citations from the retrieved documents.

The key design pattern is the *conditional loop*: after evaluation, the agent either proceeds to generation (if context is sufficient or the iteration budget is exhausted) or loops back to retrieval with refined sub-queries. This mirrors the sense–act–evaluate cycle of an RL agent operating over information-gathering actions.

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
import operator

class AgentState(TypedDict):
    query: str
    sub_queries: list[str]
    retrieved_docs: Annotated[list[dict], operator.add]
    context_sufficient: bool
    answer: str
    iterations: int
    max_iterations: int

def plan_node(state: AgentState) -> AgentState:
    """Decompose query into sub-queries."""
    sub_queries = decompose_query(state["query"])
    return {**state, "sub_queries": sub_queries, "iterations": 0}

def retrieve_node(state: AgentState) -> AgentState:
    """Retrieve documents for current sub-queries."""
    new_docs = []
    for sq in state["sub_queries"]:
        source = route_query(sq)
        docs = retrieve_from_source(sq, source)
        new_docs.extend(docs)
    return {**state, "retrieved_docs": new_docs,
            "iterations": state["iterations"] + 1}

def evaluate_node(state: AgentState) -> AgentState:
    """Evaluate whether retrieved context is sufficient."""
    sufficient = evaluate_context_sufficiency(
        query=state["query"],
        docs=state["retrieved_docs"]
    )
    return {**state, "context_sufficient": sufficient}

def generate_node(state: AgentState) -> AgentState:
    """Generate answer from retrieved context."""
    answer = generate_with_citations(
        query=state["query"],
        docs=state["retrieved_docs"]
    )
    return {**state, "answer": answer}

def should_retrieve(state: AgentState) -> str:
```

```
    if state["context_sufficient"]:
        return "generate"
    if state["iterations"] >= state["max_iterations"]:
        return "generate"  # give up and generate with what we have
    return "retrieve"

# Build the graph
workflow = StateGraph(AgentState)
workflow.add_node("plan",     plan_node)
workflow.add_node("retrieve", retrieve_node)
workflow.add_node("evaluate", evaluate_node)
workflow.add_node("generate", generate_node)

workflow.set_entry_point("plan")
workflow.add_edge("plan",     "retrieve")
workflow.add_edge("retrieve", "evaluate")
workflow.add_conditional_edges("evaluate", should_retrieve,
    {"retrieve": "retrieve", "generate": "generate"})
workflow.add_edge("generate", END)

agent = workflow.compile()

# Run
result = agent.invoke({
    "query": "What were the main causes of the 2023 banking crisis?",
    "max_iterations": 3,
    "retrieved_docs": [],
    "iterations": 0,
})
```

Listing 16.10: LangGraph-based agentic RAG

16.7.5 Tool-Augmented RAG

Agentic RAG can combine retrieval with computation tools:

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.tools import tool

@tool
def search_documents(query: str) -> str:
    """Search internal document knowledge base."""
    docs = vectorstore.similarity_search(query, k=5)
    return "\n\n".join(d.page_content for d in docs)

@tool
def query_database(sql: str) -> str:
    """Execute SQL query on the analytics database."""
    return db.run(sql)

@tool
def web_search(query: str) -> str:
    """Search the web for current information."""
    return tavily_client.search(query)

@tool
def execute_python(code: str) -> str:
    """Execute Python code for calculations."""
    return python_repl.run(code)

tools = [search_documents, query_database, web_search, execute_python]
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Listing 16.11: Tool-augmented RAG with SQL and retrieval

16.7.6 Search-R1: RL-Trained Agentic RAG

The agentic RAG approaches above rely on *prompt-engineered* orchestration — the agent's search behavior is controlled by instructions, not learned through training. **Search-R1** [170] takes a fundamentally different approach: it trains the LLM via reinforcement learning to *learn when, what, and how many times to search* as part of its reasoning process.

Core Idea. Search-R1 extends the DeepSeek-R1 [72] reasoning framework by treating search engine queries as **actions** within the RL training loop. During chain-of-thought generation, the model can emit special tokens `<search>query</search>` that trigger real-time retrieval from a search engine. The retrieved results are injected back into the reasoning context, and the model continues generating.

Formal Setup. The model generates a reasoning trace interleaved with search actions:

$$\underbrace{\text{think}_1}_{\text{reasoning}} \rightarrow \underbrace{\texttt{<search>}q_1\texttt{</search>}}_{\text{action}} \rightarrow \underbrace{[\text{results}_1]}_{\text{observation}} \rightarrow \text{think}_2 \rightarrow \texttt{<search>}q_2\texttt{</search>} \rightarrow \cdots \rightarrow \text{answer}$$

The entire trajectory (reasoning + searches + final answer) is scored by a terminal reward: correctness of the final answer against a ground-truth label.

Training Algorithm. Search-R1 uses GRPO (Group Relative Policy Optimization):

1. **Sample N trajectories** per question, each potentially containing 0–5 search calls
2. **Execute searches in real-time** — the environment returns actual search engine results
3. **Score terminal answer correctness** (exact match or F1 against ground truth)
4. **Compute group-relative advantage**: $\hat{A}_i = (R_i - \mu_G)/\sigma_G$
5. **Update policy** with GRPO clipped objective — reinforcing trajectories that searched effectively

The model learns to:

- **Search when uncertain** — avoid unnecessary searches for knowledge it already has
- **Formulate effective queries** — learn query phrasing that returns relevant results
- **Search multiple times** — iteratively refine queries based on initial results
- **Integrate retrieved context** — use search results to support or correct its reasoning

Table 16.4: Search-R1 (RL-trained) vs. prompt-based Agentic RAG.

Dimension	Prompt-Based Agentic RAG	Search-R1
Search decision	Prompt/heuristic	Learned via RL
Query formulation	Prompted ("rewrite query")	Trained end-to-end
# searches	Fixed or LLM-decided at inference	Learned optimal count
Training signal	None (frozen model)	Correctness reward
Search integration	Append to context	Interleaved in CoT
Failure recovery	Retry heuristics	Learned backoff/reformulation
Overhead at inference	Framework overhead (LangGraph)	Native model behavior

How Search-R1 Differs from Prompt-Based Agentic RAG.

Results. On open-domain QA benchmarks (NQ [194], TriviaQA [173], HotpotQA [405]), Search-R1 with a 7B model outperforms:

- Standard RAG (single retrieval) by 15–20% accuracy
- Prompted agentic RAG (ReAct-style) by 8–12% accuracy
- Approaches the performance of much larger models (70B) with standard RAG

The key insight: **learning when and how to search is more valuable than having a larger model that knows more**. A small model that searches well beats a large model that doesn't search.

Search-R1: The Paradigm Shift

Traditional RAG asks: "Given this query, what should I retrieve?" (a pipeline decision made before generation).
Search-R1 asks: "Given what I've reasoned so far, do I need more information? If so, what specific question would fill this gap?" (a learned decision made *during* generation).
This is the difference between a student who looks up the textbook before starting an exam, versus one who consults references mid-problem when they realize they're stuck. The latter is more efficient and more targeted.

16.8 Evaluation

Evaluating a RAG system is harder than evaluating retrieval or generation in isolation, because errors can originate at *any stage* of the pipeline—and they compound. A perfect generator cannot compensate for irrelevant retrievals, and a perfect retriever is wasted if the generator hallucinates or ignores the context.

Effective RAG evaluation therefore operates at **three levels**:

1. **Retrieval quality**: Did the retriever surface the right passages? (Recall, Precision, MRR, NDCG)
2. **Generation quality**: Is the answer correct, faithful to the retrieved context, and complete? (Correctness, Faithfulness, Answer Relevance)
3. **End-to-end quality**: Does the full system satisfy the user? (Human preference, task success rate, latency-adjusted utility)

A common failure mode is optimizing only one level—for example, maximizing Recall@K with large K fills the context with marginally relevant passages that actually *degrade* generation quality. The metrics below cover both retrieval and generation, enabling practitioners to diagnose which stage is the bottleneck.

16.8.1 Retrieval Metrics

Let $\mathcal{R}_k$ be the set of retrieved documents at rank k, and $\mathcal{R}^*$ be the set of relevant documents.

Recall@K.

$$\text{Recall@}K = \frac{|\mathcal{R}_K \cap \mathcal{R}^*|}{|\mathcal{R}^*|} \tag{16.22}$$

Precision@K.

$$\text{Precision@}K = \frac{|\mathcal{R}_K \cap \mathcal{R}^*|}{K} \tag{16.23}$$

Mean Reciprocal Rank (MRR).

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \tag{16.24}$$

where rank_i is the rank of the first relevant document for query i.

Normalized Discounted Cumulative Gain (NDCG@K).

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad \text{DCG@}K = \sum_{i=1}^{K} \frac{\text{rel}_i}{\log_2(i+1)} \tag{16.25}$$

where $\text{rel}_i \in \{0, 1, 2, \ldots\}$ is the graded relevance of the i-th result and IDCG is the ideal (perfect) DCG.

16.8.2 Generation Metrics

Faithfulness. Measures whether the generated answer is *grounded* in the retrieved context—i.e., every claim in the answer can be attributed to a retrieved document. Evaluated by an LLM judge:

$$\text{Faithfulness} = \frac{\#\text{ claims supported by context}}{\#\text{ total claims in answer}} \tag{16.26}$$

Answer Relevance. Measures whether the answer addresses the question. Computed by generating questions from the answer and measuring similarity to the original query:

$$\text{AnswerRelevance} = \frac{1}{N} \sum_{i=1}^{N} \cos(E(q), E(\hat{q}_i)) \tag{16.27}$$

where $\hat{q}_i$ are questions generated from the answer.

Context Precision and Recall.

$$\text{ContextPrecision@}K = \frac{1}{K} \sum_{k=1}^{K} \text{Precision@}k \cdot \mathbf{1}[\text{doc}_k \text{ is relevant}] \tag{16.28}$$

$$\text{ContextRecall} = \frac{\#\text{ ground-truth claims attributable to context}}{\#\text{ total ground-truth claims}} \tag{16.29}$$

16.8.3 RAGAs Framework

RAGAs (Retrieval Augmented Generation Assessment) [85] provides a reference-free evaluation framework using LLM judges:

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall,
    answer_correctness,
)
from datasets import Dataset

eval_dataset = Dataset.from_dict({
    "question":  questions,
    "answer":    generated_answers,
    "contexts":  retrieved_contexts,   # list of lists
    "ground_truth": reference_answers,
```

```
})

results = evaluate(
    dataset=eval_dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_precision,
        context_recall,
        answer_correctness,
    ],
)
print(results.to_pandas())
```

Listing 16.12: RAGAs evaluation (v0.1 API; v0.2+ uses `user_input`, `response`, `retrieved_contexts`, `reference`)

16.8.4 Common Failure Modes

RAG Failure Modes to Monitor

1. **Retrieval Miss**: The relevant document exists in the corpus but is not retrieved. Causes: poor chunking, embedding model mismatch, query-document vocabulary gap.
2. **Context Poisoning**: Retrieved documents contain misleading or contradictory information that causes the model to generate incorrect answers.
3. **Lost-in-the-Middle**: LLMs attend more strongly to the beginning and end of long contexts; relevant information in the middle may be ignored [225].
4. **Over-Retrieval**: Too many retrieved chunks dilute the relevant signal and increase latency and cost.
5. **Hallucination Despite Retrieval**: Model ignores retrieved context and generates from parametric memory, especially when context contradicts training data.
6. **Citation Fabrication**: Model attributes claims to documents that do not support them.

16.9 Production Considerations

16.9.1 Embedding Model Selection

The embedding model is the single most impactful component choice in a RAG system—it determines the quality ceiling for retrieval. The field has advanced rapidly; Table 16.5 summarizes current options across the cost–quality spectrum.

Table 16.5: Embedding models for production RAG (as of 2026). MTEB scores are overall averages across retrieval, classification, clustering, and STS tasks.

Model	Dims	Max Tokens	MTEB Avg	Access	Notes
API-based (managed)					
Voyage `voyage-4-large`	1024*	32K	—	API	Best retrieval quality
OpenAI `text-embedding-3-large`	3072	8191	64.6	API	Matryoshka dims
Cohere `embed-english-v3.0`	1024	512	64.5	API	int8/binary support
Google `text-embedding-005`	768	2048	—	API	Vertex AI integration
Open-weight (self-hosted)					
`nvidia/NV-Embed-v2` [202]	4096	32K	72.3	Free	#1 MTEB (Sep 2024)
`Alibaba-NLP/gte-Qwen2-7B` [213]	3584	32K	70.2	Free	Apache-2.0, multilingual
`BAAI/bge-m3` [45]	1024	8192	65.0	Free	Dense + sparse + multi-vec
`jinaai/jina-embeddings-v3`	1024	8192	66.0	Free	Multilingual, LoRA adapters
`BAAI/bge-large-en-v1.5` [395]	1024	512	64.2	Free	Mature, well-supported

Selection Criteria.

- **Domain match**: Specialized models (e.g., `voyage-code-3` for code, `voyage-finance-2` for finance) can outperform general models by 5–15% on domain tasks.
- **Context length**: Models with 32K token context (Voyage-4, NV-Embed-v2) can embed entire documents without chunking, simplifying the pipeline.
- **Matryoshka embeddings**: Models supporting flexible output dimensions (256–4096) let you trade quality for storage/latency at serving time without re-encoding.
- **Quantization support**: int8 or binary quantization at the model level (Cohere, Voyage) reduces index size by 4–32× with minimal recall loss.
- **Multilingual**: For non-English or cross-lingual RAG, prefer models explicitly trained multilingual (BGE-M3, Jina-v3, Voyage-4).

16.9.2 Vector Database Comparison

Table 16.6: Vector database comparison for production RAG systems

Database	Hosting	Scale	Filtering	Hybrid	Best For
FAISS1	Self-hosted	Billions	Limited	No	Research, offline
Pinecone2	Managed	Billions	Yes	Yes	Serverless, easy setup
Weaviate3	Both	Billions	Yes	Yes	GraphQL, multi-modal
Chroma4	Self-hosted	Millions	Yes	No	Local dev, prototyping
Qdrant5	Both	Billions	Yes	Yes	High performance
Milvus6	Both	Billions	Yes	Yes	Enterprise, GPU accel.
pgvector7	Self-hosted	Millions	Yes	Yes	Existing Postgres users

5https://qdrant.tech 6https://milvus.io 7https://github.com/pgvector/pgvector

16.9.3 Latency Optimization

1. **Pre-filtering**: Use metadata filters (date range, category, source) to reduce the search space before ANN search
2. **Approximate NN**: Use HNSW or IVF indices instead of exact search; accept ∼1% recall loss for 10× speedup
3. **Embedding caching**: Cache embeddings for frequently repeated queries
4. **Async retrieval**: Retrieve from multiple sources in parallel
5. **Streaming generation**: Stream LLM output while retrieval completes
6. **Quantization**: Use int8 or binary quantization for embeddings to reduce memory and increase throughput

Async Parallel Retrieval. Techniques (3) and (4) above compose naturally: cache the query embedding, then fan out retrieval requests to multiple backends concurrently. In a multi-source RAG system (Section 16.7), the user query may need results from a vector database, a keyword index, and a web API. Sequential retrieval adds latencies; parallel retrieval pays only the cost of the *slowest* source. Listing **??** demonstrates this pattern using Python's `asyncio`—the `lru_cache` decorator ensures repeated queries skip the embedding model entirely, while `asyncio.gather` dispatches all source queries simultaneously.

```
import asyncio
from functools import lru_cache

@lru_cache(maxsize=1024)
```

```python
def get_cached_embedding(text: str) -> list[float]:
    return embedding_model.embed_query(text)

async def parallel_retrieve(
    query: str,
    sources: list[str],
    k: int = 5
) -> list[dict]:
    """Retrieve from multiple sources in parallel."""
    tasks = [
        asyncio.create_task(retrieve_from_source_async(query, src, k))
        for src in sources
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)
    # Flatten and deduplicate
    all_docs = []
    for r in results:
        if not isinstance(r, Exception):
            all_docs.extend(r)
    return deduplicate_by_content(all_docs)
```

Listing 16.13: Async parallel retrieval for low latency

16.9.4 Incremental Indexing and Versioning

In production, the document corpus is never static—policies get revised, new reports land daily, deprecated content must be removed. A full re-index (re-chunk, re-embed, re-upload) is expensive and causes downtime. Incremental indexing solves this by applying changes at the document level.

Core Operations.

- **Upsert**: When a document is created or updated, delete all existing chunks for that `doc_id`, re-chunk the new content, embed, and insert. This guarantees no stale fragments linger.
- **Delete/Expire**: Remove chunks by document ID (explicit deletion) or by TTL (automatic garbage collection for time-sensitive sources like news or market data).
- **Version tracking**: Store a `version` and `indexed_at` timestamp in chunk metadata. This enables rollback (restore previous version from source) and auditability ("which version did the model see?").

Consistency Challenges.

- **Embedding model drift**: If you upgrade the embedding model, old and new vectors are incompatible. Solutions: (a) maintain separate indices per model version and migrate in the background, or (b) use Matryoshka-compatible models where dimension truncation preserves compatibility.
- **Chunk boundary shifts**: Changing the chunking strategy invalidates all existing chunks. Version metadata lets you identify and selectively re-index affected documents.
- **Eventual consistency**: In distributed vector databases, newly upserted vectors may not be immediately searchable. Design your pipeline to tolerate a brief indexing lag (typically seconds to minutes).

Implementation. Listing 16.14 shows a minimal `RAGIndexManager` class that encapsulates upsert and expiration logic, suitable for wrapping any vector store with metadata filtering support.

```python
class RAGIndexManager:
    def __init__(self, vectorstore, metadata_store, chunker, embedder):
        self.vs = vectorstore
        self.meta = metadata_store
        self.chunker = chunker
        self.embedder = embedder

    def upsert_document(self, doc_id: str, content: str,
                        metadata: dict) -> None:
        """Add or update a document, replacing old chunks."""
        # Delete existing chunks for this document
        self.vs.delete(filter={"doc_id": doc_id})
        # Chunk new version (vectorstore embeds internally)
        chunks = self.chunker.split_text(content)
        self.vs.add_texts(
            texts=chunks,
            metadatas=[{**metadata, "doc_id": doc_id,
                        "version": metadata.get("version", 1),
                        "indexed_at": datetime.utcnow().isoformat()}
                       for _ in chunks],
        )

    def expire_old_documents(self, ttl_days: int = 365) -> int:
        """Remove documents older than TTL."""
        cutoff = (datetime.utcnow() - timedelta(days=ttl_days)).isoformat()
        return self.vs.delete(filter={"indexed_at": {"$lt": cutoff}})
```

Listing 16.14: Incremental index updates with versioning

16.10 RAG + Fine-Tuning Synergy

16.10.1 When to Combine RAG with Fine-Tuning

Fine-tuning and RAG address complementary weaknesses:

- **Fine-tuning alone**: model learns style and format but may hallucinate facts
- **RAG alone**: model has access to facts but may not know how to use them optimally
- **Combined**: fine-tune the model to *use retrieved context well*—cite sources, acknowledge uncertainty, and ignore irrelevant context

16.10.2 RAFT: Retrieval-Augmented Fine-Tuning

RAFT [425] trains models to answer questions given a mix of relevant and *distractor* documents, teaching the model to identify and use only the relevant context:

1. For each training example (q, a, d^*), sample $k-1$ distractor documents $\{d_i^-\}$
2. Fine-tune on: `[q,` d^*`,` d_1^-`, ...,` d_{k-1}^-`]` $\rightarrow$ `[chain-of-thought + a]`
3. The chain-of-thought explicitly quotes from d^*, teaching the model to ground answers

$$\mathcal{L}_{\text{RAFT}} = -\mathbb{E}_{(q,a,d^*,\{d_i^-\})}\left[\log P_\theta\Big(\text{CoT}(d^*) \oplus a \;\Big|\; q, d^*, \{d_i^-\}\Big)\right] \tag{16.30}$$

16.10.3 Joint Retriever-Generator Training

For maximum performance, the retriever and generator can be trained jointly. The REALM [128] and RAG [208] papers propose end-to-end training where gradients flow through the retrieval step:

$$\nabla_\theta \mathcal{L} = \nabla_\theta \left[-\log \sum_{d \in \mathcal{D}} P_\theta(a \mid q, d) \cdot P_\phi(d \mid q) \right] \tag{16.31}$$

The retriever parameters ϕ are updated using the REINFORCE estimator or by treating $P_\phi(d \mid q)$ as a differentiable attention over documents.

Joint Training Challenges

Joint retriever-generator training is powerful but complex: (1) the document index must be periodically refreshed as ϕ changes (asynchronous index refresh), (2) the training signal is sparse (only top-k documents contribute), and (3) training is unstable without careful initialization from a pre-trained retriever.

16.11 Comprehensive RAG Approach Comparison

Table 16.7: RAG approaches across key dimensions

Approach	Accuracy	Latency	Complexity	Cost	Best For
Naive RAG [208]	Medium	Low	Low	Low	Prototyping, simple QA
RAG + Re-ranking [268]	High	Medium	Medium	Medium	Production QA systems
HyDE [106]	High	Medium	Low	Medium	Semantic mismatch domains
Multi-Query RAG	High	Medium	Medium	Medium	Ambiguous queries
RAG-Fusion [300]	High	Medium	Medium	Medium	Diverse query types
Self-RAG [13]	High	Medium	High	Medium	Selective retrieval
CRAG [401]	High	Medium	High	High	Unreliable corpora
Adaptive RAG [162]	High	Low–High	High	Medium	Mixed query complexity
Graph RAG [82]	V. High	High	V. High	High	Global synthesis queries
Agentic RAG	V. High	High	V. High	High	Multi-hop reasoning
RAFT [425]	V. High	Low	V. High	V. High	Domain-specific deployment

Key Design Questions for RAG Systems

When designing a RAG system for production, consider:

1. **What is the query distribution?** Factoid vs. analytical vs. multi-hop queries require different retrieval strategies.
2. **How large and dynamic is the corpus?** Millions of documents with frequent updates favor managed vector databases with incremental indexing.
3. **What are the latency requirements?** Sub-100ms responses preclude re-ranking and agentic loops; batch or async use cases can afford them.
4. **How critical is grounding?** High-stakes domains (medical, legal, financial) require faithfulness evaluation and citation verification.
5. **Is the vocabulary specialized?** Domain-specific terminology may require hybrid retrieval or domain-adapted embedding models.

RAG Best Practices Summary

- **Start simple**: naive RAG with good chunking often outperforms complex systems with poor chunking
- **Evaluate retrieval separately**: fix retrieval before optimizing generation
- **Use hybrid retrieval**: BM25 + dense with RRF is a strong default